

**LAPORAN FINAL PROJECT SISTEM
MANAJEMEN PERPUSTAKAN DIGITAL**

Dosen Pengampu : Taufik Ridwan, MT



Disusun Oleh:

Aqila Al Jundy (2510631250053)

Khalid Abdulmalik (2510631250065)

Marsya Ayu Azzahro (2510631250014)

Mata Kuliah: Algoritma & Struktur Data

Fakultas Ilmu Komputer

Program Studi Sistem Informasi

Universitas Singaperbangsa Karawang

2026

DAFTAR ISI

BAB 1	4
PENDAHULUAN.....	4
1.1 Latar Belakang Masalah	4
1.2 Identifikasi Masalah.....	5
1.3 Batasan Masalah.....	5
1.4 Tujuan Projek Akhir.....	6
1.5 Manfaat Sistem.....	6
BAB 2	7
TINJAUAN PUSTAKA DAN LANDASAN TEORI.....	7
2.1 Konsep Struktur Data Dinamis dan ArrayList.....	7
2.2 Teori Algoritma Pengurutan (Sorting) Bubble Sort	7
2.3 Teori Algoritma Pencarian (Searching) Linear Search	8
2.4 Analisis Kompleksitas Algoritma (Notasi Big-O)	9
2.5 Sistem Manajemen Basis Data Berbasis File Flat (CSV).....	9
BAB 3	11
ANALISIS KEBUTUHAN DAN DESAIN SISTEM	11
3.1 Analisis Kebutuhan Fungsional (Multi-role Administrasi)	11
3.2 Spesifikasi Data dan Array of Records Layout.....	11
3.3 Perancangan Logika Alur Sistem (Flowchart).....	13
BAB 4	14
IMPLEMENTASI SISTEM DAN BEDAH KODE	14
4.1 Struktur Kelas Utama dan Inisialisasi Koleksi Data.....	14
4.2 Fungsi Manajemen Data Buku (CRUD Buku).....	15
4.3 Fungsi Manajemen Data Anggota (CRUD Anggota)	16
4.4 Fungsi Transaksi Peminjaman dan Pengembalian	17
4.5 Implementasi Detak Algoritma Bubble Sort Manual	18

4.6 Implementasi Detak Algoritma Linear Search Manual	19
4.7 Mekanisme Read/Write Data File CSV Persisten	20
BAB 5	22
ANALISIS KOMPLEKSITAS DAN PERFORMA	22
5.1 Pembuktian Kompleksitas Waktu Bubble Sort.....	22
5.2 Pembuktian Kompleksitas Waktu Linear Search	22
5.3 Analisis Efisiensi Penggunaan Memori (Space Complexity).....	23
BAB 6	24
PENGUJIAN SISTEM DAN SKENARIO	24
6.1 Skenario Pengujian Operasi CRUD dan Validasi Bisnis	24
6.2 Skenario Pengujian Sorting Multi-kolom.....	27
6.3 Skenario Pengujian Searching Substring Matching.....	29
6.4 Skenario Pengujian Kegagalan Sistem (Error Handling)	31
BAB 7	34
KESIMPULAN DAN SARAN	34
7.1 Kesimpulan.....	34
7.2 Saran dan Pengembangan Kedepan	34

BAB 1

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi informasi pada era globalisasi saat ini telah mengubah paradigma pengelolaan data di berbagai instansi, tidak terkecuali institusi pendidikan seperti perpustakaan universitas atau sekolah. Perpustakaan, sebagai pusat retensi dan distribusi pengetahuan, menyimpan volume informasi yang sangat masif berupa koleksi buku, jurnal, majalah, serta dokumen akademis lainnya. Manajemen operasional konvensional yang mengandalkan pencatatan berbasis kertas atau pembukuan manual terbukti memicu timbulnya degradasi performa pelayanan, inefisiensi waktu, rentannya manipulasi data tidak sah, tingginya probabilitas kesalahan manusia (human error), serta kesulitan dalam melakukan audit keuangan dan pelacakan inventaris.

Di Universitas Singaperbangsa Karawang (UNSIKA), pemahaman mahasiswa mengenai integrasi fungsional antara teori akademis struktur data dengan kebutuhan industri di lapangan menjadi sangat krusial. Salah satu masalah fundamental dalam administrasi perpustakaan konvensional adalah lambatnya proses pencarian buku. Ketika seorang anggota ingin meminjam buku, pustakawan harus memeriksa katalog fisik satu per satu. Hal ini tidak hanya memakan waktu tetapi juga mengurangi produktivitas. Oleh karena itu, sebuah perangkat lunak yang andal untuk mengotomatisasi seluruh alur bisnis perpustakaan sangat dibutuhkan.

Aplikasi "Sistem Perpustakaan Digital" yang dikembangkan menggunakan bahasa pemrograman Java ini hadir sebagai sebuah solusi holistik. Perangkat lunak ini tidak sekadar bertindak sebagai antarmuka pencatatan data, melainkan mengintegrasikan berbagai konsep fundamental ilmu komputer seperti enkapsulasi objek, array of records dinamis via `ArrayList`, algoritma pengurutan data leksikografis internal, serta mesin pencarian string sekuensial. Seluruh aspek teknis ini diimplementasikan tanpa memanfaatkan pustaka utilitas bawaan luar untuk menjamin pemahaman mendalam mengenai efisiensi eksekusi tingkat rendah.

1.2 Identifikasi Masalah

Berdasarkan deskripsi kondisi riil lapangan yang melatarbelakangi proyek ini, dapat diidentifikasi beberapa poin permasalahan teknis sebagai berikut:

1. Bagaimana cara mengorganisasikan entitas data terstruktur seperti Buku, Anggota, dan Transaksi Peminjaman di dalam memori komputer menggunakan prinsip linear data structure secara optimal tanpa menyebabkan pemborosan alokasi memori?
2. Bagaimana merancang sistem manipulasi data yang aman (CRUD) yang dilengkapi mekanisme pencegahan data duplikat (seperti kesamaan ISBN) serta penanganan penghapusan data relasional (foreign key constraints manual)?
3. Bagaimana menyusun algoritma pengurutan data leksikografis string (Bubble Sort) secara mandiri untuk mengurutkan data buku berdasarkan judul, pengarang, maupun stok terkini tanpa memanggil fungsi `Collections.sort()`?
4. Bagaimana mengimplementasikan pencarian data (Linear Search) dengan fitur substring matching yang responsif untuk mencari data buku secara fleksibel berdasarkan masukan pengguna yang bersifat case-insensitive?
5. Bagaimana merancang model penyimpanan data yang persisten berskala ringan menggunakan flat file CSV agar seluruh state data tidak hilang ketika runtime Java Virtual Machine (JVM) dihentikan?

1.3 Batasan Masalah

Agar pembahasan dalam laporan proyek akhir ini tetap fokus, terarah, dan mendalam, penulis membatasi ruang lingkup pengembangan sistem sebagai berikut:

- **Bahasa Pemrograman:** Sistem dibangun secara eksklusif menggunakan Java Standard Edition (Java SE) versi 17 atau di atasnya.
- **Arsitektur Struktur Data:** Penyimpanan memori utama bertumpu pada struktur data linear `List<String[]>` (Array of Strings di dalam objek `ArrayList`) yang merepresentasikan model tabel basis data relasional secara logis.
- **Batasan Algoritma:** Algoritma pengurutan dibatasi pada metode *Bubble Sort* tradisional. Algoritma pencarian dibatasi pada metode *Linear Search* konvensional dengan pemindaian sekuensial dari awal hingga akhir elemen data.

- **Persistensi Basis Data:** Tidak menggunakan sistem manajemen basis data relasional (RDBMS) komersial seperti MySQL atau PostgreSQL, melainkan menggunakan tiga file flat CSV lokal (`databuku.csv`, `dataanggota.csv`, dan `datapinjam.csv`).
- **Antarmuka Pengguna:** Aplikasi berbasis antarmuka baris perintah atau Command Line Interface (CLI) yang dijalankan melalui console terminal interaktif.

1.4 Tujuan Proyek Akhir

Tujuan utama dari pengerjaan proyek akhir ini terbagi menjadi dua aspek, yaitu tujuan instruksional akademis dan tujuan fungsional sistemik:

Secara akademis, proyek ini bertujuan untuk membuktikan kompetensi mahasiswa Informatika UNSIKA dalam menerapkan teori struktur data linier dinamis dan analisis kompleksitas Big-O ke dalam implementasi praktis. Proyek ini memvalidasi pemahaman mahasiswa mengenai cara kerja internal penukaran memori (swapping) pada algoritma sortasi dasar serta penelusuran alamat array secara sekuensial tanpa bergantung pada framework pihak ketiga.

Secara fungsional sistemik, proyek ini memproduksi sebuah prototipe perangkat lunak manajemen perpustakaan digital yang siap pakai, aman, dan memiliki alur kerja yang logis. Sistem ini mampu mengelola hak akses berlapis (Admin dan User), menjaga konsistensi stok buku secara otomatis saat transaksi peminjaman berlangsung, serta menyediakan data analisis inventaris perpustakaan yang valid.

1.5 Manfaat Sistem

Aplikasi ini diharapkan dapat memberikan kontribusi signifikan bagi berbagai pihak, di antaranya:

Bagi Pustakawan dan Administrator Perpustakaan, aplikasi ini memangkas waktu birokrasi administrasi secara drastis. Proses penambahan buku baru, pembaruan biodata anggota, hingga pengecekan status peminjaman yang terlambat dapat diselesaikan dalam hitungan detik melalui terminal perintah tanpa perlu membolak-balik lembar dokumen fisik.

Bagi Anggota Perpustakaan (User), sistem menyediakan transparansi informasi katalog yang akurat. Pengguna dapat mencari tahu ketersediaan stok buku tertentu secara mandiri dari terminal, melihat daftar peminjaman aktif mereka sendiri, serta melakukan simulasi pengembalian secara tertib tanpa risiko kesalahan pencatatan oleh petugas.

BAB 2

TINJAUAN PUSTAKA DAN LANDASAN TEORI

2.1 Konsep Struktur Data Dinamis dan ArrayList

Struktur data merupakan cara mengorganisasikan, menyimpan, dan merepresentasikan data di dalam memori komputer agar dapat diakses dan dimanipulasi secara efisien. Secara umum, struktur data linier dibagi menjadi dua kategori utama, yaitu struktur data statis (seperti array konvensional) dan struktur data dinamis (seperti `LinkedList` atau `ArrayList`). Array statis memiliki kelemahan fatal berupa ukuran alokasi memori yang bersifat tetap (*fixed-size*) semenjak deklarasi awal, sehingga membatasi skalabilitas aplikasi apabila volume data di lapangan melonjak melampaui batas alokasi tersebut.

Untuk mengatasi keterbatasan array statis, bahasa pemrograman Java menyediakan kelas utilitas `java.util.ArrayList` yang mengimplementasikan antarmuka (interface) `java.util.List`. Secara arsitektural internal, `ArrayList` bertumpu pada array statis objek yang dapat diubah ukurannya secara dinamis (*resizable array*). Ketika jumlah elemen yang dimasukkan ke dalam `ArrayList` telah mencapai batas kapasitas maksimum (*threshold capacity*), runtime Java secara otomatis akan mengalokasikan array baru dengan kapasitas yang lebih besar (biasanya berkembang sebesar $1.5 \times$ dari ukuran sebelumnya), menyalin seluruh elemen lama ke array baru tersebut, dan menghapus array lama melalui mekanisme *Garbage Collection*.

Dalam proyek Sistem Perpustakaan Digital ini, sebuah entitas record diwakili oleh larik string (`String[]`), di mana setiap indeks elemen merepresentasikan kolom atribut tertentu. Struktur koleksi akhir dibentuk melalui deklarasi `List<String[]>`. Desain arsitektur pengarsipan memori ini memiliki efisiensi tinggi untuk operasi penelusuran indeks, karena waktu akses acak (*random access time*) pada `ArrayList` bersifat konstan atau bernilai $O(1)$ berkat pemanfaatan memori berurutan (*contiguous memory allocation*).

2.2 Teori Algoritma Pengurutan (Sorting) Bubble Sort

Pengurutan data atau *sorting* merupakan proses mengatur sekumpulan data ke dalam urutan logis tertentu, baik secara menaik (*ascending*) dari nilai terkecil ke nilai terbesar, maupun menurun (*descending*) dari nilai terbesar ke nilai terkecil. Berbagai macam algoritma sortasi telah dikembangkan dalam ilmu komputer, mulai dari yang sederhana seperti *Bubble Sort*, *Selection Sort*, dan *Insertion Sort*, hingga yang kompleks seperti *Merge Sort* dan *Quick Sort*.

Algoritma *Bubble Sort* (pengurutan gelembung) merupakan salah satu algoritma sorting paling elementer. Logika dasar dari algoritma ini adalah membandingkan dua elemen data yang saling berdampingan secara berurutan sepanjang koleksi data dari indeks awal hingga indeks akhir. Jika elemen kiri memiliki nilai lebih besar daripada elemen kanan (pada skenario ascending), kedua elemen tersebut akan ditukar (swapping). Proses pemindaian ini dilakukan berulang kali dalam beberapa lintasan (passes). Pada setiap akhir lintasan, elemen dengan nilai terbesar akan "terapung" atau menempati posisi paling ujung kanan, layaknya sebuah gelembung sabun di dalam air.

Meskipun *Bubble Sort* mudah untuk dipahami secara konseptual dan mudah diimplementasikan ke dalam baris kode tanpa ketergantungan library, algoritma ini memiliki kelemahan performa yang sangat signifikan pada volume data skala besar. Hal ini disebabkan karena jumlah operasi perbandingan elemen yang masif tanpa adanya kecerdasan adaptasi terhadap kondisi awal data, kecuali jika ditambahkan variabel bendera (flag optimization) untuk menghentikan loop lebih awal jika tidak ada lagi pertukaran data yang terjadi dalam satu lintasan penuh.

2.3 Teori Algoritma Pencarian (Searching) Linear Search

Pencarian data (searching) adalah proses menemukan lokasi atau keberadaan suatu elemen data tertentu (yang disebut sebagai target atau kunci pencarian) di dalam suatu kumpulan data (koleksi atau array). Algoritma pencarian secara umum dibagi menjadi dua kategori besar berdasarkan kondisi awal kumpulan data, yaitu *Linear Search* (pencarian linier/sekuensial) yang tidak mendikte prasyarat keterurutan data, dan *Binary Search* (pencarian biner) yang mewajibkan seluruh koleksi data berada dalam kondisi terurut sempurna sebelum proses pencarian dieksekusi.

Algoritma *Linear Search* bekerja dengan cara melakukan penelusuran (traversal) secara berurutan satu per satu, mulai dari elemen pada indeks pertama (indeks 0) hingga elemen pada indeks terakhir ($n - 1$) di dalam array. Pada setiap iterasi loop, nilai elemen saat ini akan dicocokkan dengan nilai kunci pencarian (target string). Jika kedua nilai tersebut cocok, algoritma akan mengembalikan status berhasil (atau mengembalikan posisi indeks elemen tersebut) dan menghentikan iterasi. Jika penelusuran telah mencapai ujung akhir array dan tidak ada satu pun elemen yang cocok, algoritma akan menyimpulkan bahwa data yang dicari tidak ditemukan dalam sistem.

Kelebihan utama dari *Linear Search* adalah fleksibilitasnya yang tinggi, karena dapat langsung diterapkan pada kumpulan data yang acak, dinamis, dan sering berubah tanpa perlu membuang resource komputasi tambahan untuk melakukan sortasi terlebih dahulu. Selain itu,

algoritma ini sangat efektif untuk pencarian parsial atau pencarian berbasis substring, di mana pengguna tidak perlu memasukkan kata kunci secara utuh (cukup beberapa karakter yang mewakili bagian dari judul buku atau nama anggota).

2.4 Analisis Kompleksitas Algoritma (Notasi Big-O)

Analisis kompleksitas algoritma merupakan sebuah metodologi matematis yang digunakan untuk mengukur efisiensi suatu algoritma seiring dengan pertumbuhan ukuran data masukan (n). Efisiensi algoritma diukur berdasarkan dua parameter utama, yaitu kompleksitas waktu (time complexity) yang mengukur seberapa cepat runtime algoritma bertambah, dan kompleksitas ruang (space complexity) yang mengukur seberapa besar kebutuhan alokasi memori tambahan yang diperlukan oleh algoritma selama proses eksekusi berlangsung.

Notasi Big-O (O) digunakan untuk mendeskripsikan batas atas (upper bound) dari waktu eksekusi atau kebutuhan memori suatu algoritma pada skenario terburuk (worst-case scenario). Melalui Notasi Big-O, konstan kuantitatif kecil dan suku-suku berderajat rendah diabaikan, sehingga fokus analisis murni tertuju pada laju pertumbuhan asimtotik dari fungsi tersebut. Sebagai contoh, sebuah algoritma dengan fungsi waktu $f(n) = 3n^2 + 5n + 10$ akan diklasifikasikan sebagai algoritma berkinerja kuadratik dengan notasi kompleksitas $O(n^2)$.

Memahami karakteristik kompleksitas asimtotik sangat penting bagi seorang insinyur perangkat lunak untuk menghindari kegagalan sistem akibat kehabisan daya komputasi CPU atau memori RAM. Sebuah fungsi bernilai $O(n)$ akan bertambah secara linear dan konstan searah dengan pertambahan data, sementara fungsi $O(n^2)$ akan mengalami pembengkakan waktu eksekusi yang eksponensial ketika data masukan menyentuh angka ribuan atau jutaan baris, seperti yang terjadi pada struktur perulangan bersarang (nested loops) tanpa optimasi khusus.

2.5 Sistem Manajemen Basis Data Berbasis File Flat (CSV)

Dalam rekayasa perangkat lunak skala kecil dan menengah, penggunaan flat file (file teks rata) sering kali menjadi pilihan utama yang paling efisien dibandingkan dengan implementasi server database relasional penuh yang membutuhkan konfigurasi jaringan dan overhead instalasi yang rumit. Salah satu format flat file yang paling populer dan diakui secara universal di berbagai platform adalah format *Comma-Separated Values* atau yang biasa disingkat sebagai CSV.

Secara struktural, sebuah file CSV menyimpan data tabel dalam bentuk baris-baris teks polos (plain text), di mana setiap baris mewakili satu record data (trow atau baris tabel), dan setiap atribut atau kolom di dalam baris tersebut dipisahkan oleh karakter pembatas spesifik, yang paling umum

adalah tanda koma (,) atau titik koma (;). Karakteristik utama dari format CSV adalah kesederhanaannya yang tinggi, kemudahan untuk dibaca secara langsung oleh manusia (human-readable), serta kompatibilitasnya yang luas untuk diimpor atau diekspor dari dan ke berbagai aplikasi spreadsheet modern seperti Microsoft Excel atau Google Sheets.

Namun, penyimpanan data berbasis CSV dalam memori persisten lokal mewajibkan pengembang untuk mengimplementasikan parser manual yang andal di dalam kode aplikasi. Proses manipulasi file melibatkan operasi input/output (I/O) stream yang cukup intensif, di mana aplikasi harus membaca seluruh isi file baris demi baris, memecah string menggunakan fungsi tokenisasi atau ekspresi reguler (seperti fungsi `String.split()` pada Java), dan mengonversinya kembali menjadi representasi objek memori sebelum dapat dimanipulasi oleh algoritma internal sistem.

BAB 3

ANALISIS KEBUTUHAN DAN DESAIN SISTEM

3.1 Analisis Kebutuhan Fungsional (Multi-role Administrasi)

Analisis kebutuhan fungsional mendefinisikan kapabilitas, layanan, dan fungsionalitas spesifik yang wajib disediakan oleh sistem perpustakaan digital ini kepada pengguna berdasarkan hak akses masing-masing. Sistem membagi pengguna ke dalam dua kategori peran (role), yaitu peran Administrator (Petugas Perpustakaan) dan peran User (Anggota/Mahasiswa), demi menjaga keamanan data dan batasan otorisasi bisnis.

Peran **Administrator** memegang otoritas penuh terhadap integritas basis data perpustakaan. Kebutuhan fungsional untuk Admin mencakup operasi CRUD Buku menyeluruh (menambah koleksi, memperbarui informasi judul/kategori, menghapus buku dari katalog), operasi CRUD Anggota (registrasi kartu anggota baru, pembaruan nomor telepon, penghapusan keanggotaan), serta pemantauan laporan transaksi peminjaman global secara real-time dari seluruh anggota terdaftar.

Peran **User (Anggota)** memiliki hak akses yang terbatas namun interaktif. User berhak melakukan pencarian buku secara mandiri dari terminal, melihat detail stok buku yang tersedia, melacak riwayat pinjaman pribadi mereka, melakukan transaksi peminjaman mandiri yang akan memotong kuota stok buku, serta mengembalikan buku yang telah selesai dibaca untuk memulihkan kapasitas stok di rak perpustakaan digital.

3.2 Spesifikasi Data dan Array of Records Layout

Untuk merepresentasikan tabel basis data relasional tanpa menggunakan engine database eksternal, sistem menggunakan konsep *Array of Records* yang diwujudkan melalui koleksi `List<String[]>`. Setiap baris data diwakili oleh array string dengan panjang tetap, di mana posisi indeks elemen bertindak sebagai nama kolom secara implisit. Penentuan tata letak indeks ini harus konsisten di seluruh bagian kode program agar tidak memicu error indeks di luar batas (`ArrayIndexOutOfBoundsException`).

Berikut adalah spesifikasi detail mengenai struktur data dan pemetaan tata letak kolom atribut untuk masing-masing dari ketiga entitas utama sistem perpustakaan:

Tabel 3.1: Tata Letak Atribut Entitas Buku (daftarBuku)

Indeks Array	Nama Atribut (Kolom)	Tipe Data Representatif	Keterangan Bisnis
0	ISBN	String (Numeric Only)	Kunci Unik Buku (Primary Key)
1	Judul Buku	String Alfanumerik	Judul Lengkap Koleksi Buku
2	Kategori / Genre	String Alfabetis	Klasifikasi (Teknologi, Fiksi, dst.)
3	Nama Pengarang	String Alfabetis	Penulis asli buku tersebut
4	Stok Buku	String (Integer Parsed)	Jumlah fisik buku yang tersedia

Tabel 3.2: Tata Letak Atribut Entitas Anggota (daftarAnggota)

Indeks Array	Nama Atribut (Kolom)	Tipe Data Representatif	Keterangan Bisnis
0	ID Anggota	String (Format: A001)	Kunci Unik Anggota (Primary Key)
1	Nama Lengkap	String Alfabetis	Nama sesuai kartu identitas resmi
2	Nomor Telepon	String (Numeric)	Kontak aktif untuk keperluan urgen

Tabel 3.3: Tata Letak Atribut Entitas Transaksi Peminjaman (daftarPinjam)

Indeks Array	Nama Atribut (Kolom)	Tipe Data Representatif	Keterangan Bisnis
0	ID Transaksi	String (Format: T001)	

Indeks Array	Nama Atribut (Kolom)	Tipe Data Representatif	Keterangan Bisnis
			Kunci Unik Transaksi (Primary Key)
1	ID Anggota	String (Format: A001)	Relasi ke Anggota (Foreign Key)
2	ISBN Buku	String (Numeric Only)	Relasi ke Buku (Foreign Key)
3	Tanggal Transaksi	String (Format: YYYY-MM-DD)	Tanggal pencatatan peminjaman
4	Status Transaksi	String Alfabetis	"Dipinjam" atau "Dikembalikan"

3.3 Perancangan Logika Alur Sistem (Flowchart)

Alur logika program dirancang sedemikian rupa agar meminimalkan crash runtime dan menjaga integritas data antarfile. Ketika aplikasi pertama kali dieksekusi, sistem masuk ke fase bootstrapping di mana modul I/O secara otomatis membaca file `databuku.csv`, `dataanggota.csv`, dan `datapinjam.csv`. Jika file-file tersebut tidak ditemukan di direktori lokal, sistem akan mengabaikan proses load tanpa menghentikan program, lalu menginisialisasi koleksi `ArrayList` kosong sebagai wadah awal.

Setelah inisialisasi selesai, program menampilkan menu login utama. Pengguna diminta memilih peran mereka. Jika memilih Admin, sistem meminta verifikasi password (jika diaktifkan) dan membuka gerbang Menu Admin yang berisi sepuluh opsi manipulasi data terstruktur. Jika pengguna memilih peran User, mereka akan diarahkan ke Menu User yang berfokus pada pencarian katalog dan peminjaman mandiri. Setiap kali Admin atau User menyelesaikan satu operasi yang mengubah state data (seperti penambahan atau penghapusan), modul auto-save akan langsung dipicu untuk menulis ulang data dari memori RAM komputer ke media penyimpanan hard disk dalam format CSV, sehingga menjamin persistensi data yang mutakhir.

BAB 4

IMPLEMENTASI SISTEM DAN BEDAH KODE

4.1 Struktur Kelas Utama dan Inisialisasi Koleksi Data

Sistem ini diimplementasikan dalam sebuah berkas tunggal yang kohesif bernama `Perpustakaan.java`. Seluruh variabel state data dan referensi objek dideklarasikan sebagai variabel statis kelas (`static class variables`) agar dapat diakses secara langsung oleh berbagai metode fungsional tanpa perlu melakukan instansiasi objek kelas utama secara berulang kali. Pendekatan ini sangat efisien untuk aplikasi berbasis CLI kecil yang bertumpu pada arsitektur prosedural-fungsional di atas fondasi orientasi objek Java.

Berikut adalah potongan kode inisialisasi kelas, pustaka input-output yang diimpor, serta deklarasi variabel global memori utama sistem:

```
import java.util.*;
import java.io.*;

public class Perpustakaan {
    // ==== DATA SIMPANAN MEMORI UTAMA ====
    static List<String[]> daftarBuku = new ArrayList<>();
    static List<String[]> daftarAnggota = new ArrayList<>();
    static List<String[]> daftarPinjam = new ArrayList<>();

    // Objek Scanner Global untuk Menerima Input Konsol
    static Scanner input = new Scanner(System.in);

    public static void main(String[] args) {
        // Memuat data dari media penyimpanan persisten saat bootstrap
        aplikasi
        bacaBukuCSV();
        bacaAnggotaCSV();
        bacaPinjamCSV();

        // Eksekusi Loop Menu Utama
        jalankanSistemUtama();
    }
    // ... Implementasi fungsi-fungsi fungsional diletakkan di bawah sini
}
```

4.2 Fungsi Manajemen Data Buku (CRUD Buku)

Operasi CRUD (Create, Read, Update, Delete) Buku merupakan pilar utama inventarisasi perpustakaan. Fungsi `tambahBuku()` dirancang dengan validasi ganda yang ketat. Sebelum array `string` buku dimasukkan ke dalam `daftarBuku`, sistem akan melakukan iterasi linear untuk memastikan bahwa ISBN yang diinput belum pernah terdaftar sebelumnya di sistem. Hal ini penting untuk mencegah ambiguitas relasi data transaksi di kemudian hari.

Berikut adalah implementasi lengkap fungsi penambahan data buku beserta validasi keunikan ISBN dan mekanisme penyimpanan instan ke file CSV:

```
353 static void editBuku() {
354     System.out.println(x: "\n--- EDIT BUKU ---");
355     System.out.print(s: "Masukkan ISBN buku yang ingin diedit: ");
356     String isbn = input.nextLine();
357     for (String[] b : daftarBuku) {
358         if (b[0].equals(isbn)) {
359             System.out.print(s: "Judul baru   : ");
360             b[1] = input.nextLine();
361             System.out.print(s: "Kategori baru : ");
362             b[2] = input.nextLine();
363             System.out.print(s: "Pengarang baru: ");
364             b[3] = input.nextLine();
365             System.out.print(s: "Stok baru     : ");
366             // rapihin
367             String stokBaru = input.nextLine();
368
369             try {
370                 Integer.parseInt(stokBaru);
371                 b[4] = stokBaru;
372             } catch (NumberFormatException e) {
373                 System.out.println(x: "Stok harus berupa angka!");
374                 return;
375             }
376             System.out.println(x: "Buku berhasil diedit!");
377             simpanBukuCSV();
378             return;
379         }
380     }
381     System.out.println(x: "ISBN tidak ditemukan!");
382 }
```

```

// Memasukkan record ke dalam koleksi ArrayList
String[] bukuBaru = {isbn, judul, kategori, pengarang, stok};
daftarBuku.add(bukuBaru);

// Sinkronisasi Persistensi Data
simpanBukuCSV();

System.out.println("[✓] Berhasil: Buku " + judul + " telah disimpan ke
sistem.");
}

```

4.3 Fungsi Manajemen Data Anggota (CRUD Anggota)

Manajemen anggota mengadopsi pola pencatatan data yang serupa dengan entitas buku. Namun, untuk mempermudah identifikasi unik anggota tanpa mewajibkan input nomor kartu yang panjang, sistem mengimplementasikan logika generator ID otomatis (Auto-increment ID). Sistem akan memeriksa ukuran elemen saat ini di dalam `daftarAnggota` atau membaca record terakhir untuk menentukan nomor urut anggota berikutnya, misalnya "A001", "A002", dan seterusnya.

Proses penghapusan data anggota (`hapusAnggota()`) juga dilengkapi dengan validasi relasional (foreign key check manual). Seorang anggota tidak diizinkan untuk dihapus dari basis data apabila yang bersangkutan masih memiliki catatan transaksi aktif dengan status "Dipinjam" di dalam koleksi `daftarPinjam`. Aturan bisnis ini wajib ditegakkan untuk menghindari data yatim (orphan data) yang dapat merusak akurasi laporan statistik perpustakaan.

```

628 static void hapusAnggota() {
629     System.out.println(x: "\n--- HAPUS ANGGOTA ---");
630     System.out.print(s: "Masukkan ID anggota: ");
631     String id = input.nextLine();
632     for (String[] p : daftarPinjam) {
633         if (p[1].equals(id) && p[4].equals(anObject: "Dipinjam")) {
634             System.out.println(x: "Anggota masih memiliki pinjaman aktif!");
635             return;
636         }
637     }
638     boolean dihapus = daftarAnggota.removeIf(a -> a[0].equals(id));
639     System.out.println(dihapus ? "Anggota berhasil dihapus!" : "ID tidak ditemukan!");
640
641     simpanAnggotaCSV();
642 }
643

```


4.4 Fungsi Transaksi Peminjaman dan Pengembalian

Transaksi peminjaman menguji integrasi relasional antar entitas data di dalam memori utama. Ketika fungsi `pinjamBuku()` dieksekusi, program akan meminta input ID Anggota dan ISBN Buku. Sistem kemudian melakukan pencarian linear ganda: memastikan ID Anggota valid dan memastikan ISBN Buku terdaftar. Jika kedua kondisi terpenuhi, sistem melangkah ke validasi ketiga, yaitu memeriksa apakah kolom stok buku (indeks 4) bernilai lebih besar dari nol.

Jika stok tersedia, program akan mengurangi nilai integer stok tersebut sebanyak satu satuan, mengonversinya kembali menjadi string, dan memperbarui record di dalam `daftarBuku`. Selanjutnya, sebuah record transaksi baru dibuat dengan status "Dipinjam" dan dimasukkan ke dalam `daftarPinjam`.

```
684 static void kembalikanBuku() {
685     System.out.println(x: "\n--- KEMBALIKAN BUKU ---");
686     System.out.print(s: "ID Peminjaman: ");
687     String idPinjam = input.nextLine();
688
689     for (String[] p : daftarPinjam) {
690         if (p[0].equals(idPinjam)) {
691             if (p[4].equals(anObject: "Dikembalikan")) {
692                 System.out.println(x: "Buku sudah pernah dikembalikan!");
693                 return;
694             }
695             p[4] = "Dikembalikan";
696             for (String[] b : daftarBuku) {
697                 if (b[0].equals(p[2])) {
698                     b[4] = String.valueOf(Integer.parseInt(b[4]) + 1);
699                     break;
700                 }
701             }
702             System.out.println(x: "Buku berhasil dikembalikan!");
703             System.out.println("Tanggal Kembali: " + java.time.LocalDate.now());
704             System.out.println("ID Peminjaman: " + idPinjam);
705             System.out.println("ID Anggota: " + p[1]);
706             System.out.println("ISBN Buku: " + p[2]);
707             simpanBukuCSV(); // Simpan perubahan stok
708             simpanPinjamCSV(); // Simpan perubahan status peminjaman
709             return;
710         }
711     }
712     System.out.println(x: "ID Peminjaman tidak ditemukan!");
713 }
```

```
729 static void riwayatAnggota() {
730     System.out.println(x: "\nMasukkan ID anggota: ");
731     String id = input.nextLine();
732
733     boolean anggotaAda = false;
734     for (String[] a : daftarAnggota) {
735         if (a[0].equals(id)) {
736             anggotaAda = true;
737             System.out.println("Riwayat peminjaman untuk anggota: " + a[1]);
738             break;
739         }
740     }
741     if (!anggotaAda) {
742         System.out.println(x: "Anggota tidak ditemukan!");
743         return;
744     }
745
746     boolean ketemu = false;
747     for (String[] p : daftarPinjam) {
748         if (p[1].equalsIgnoreCase(id)) {
749             System.out.printf(format: "%-8s %-8s %-15s %s\n", p[0], p[2], p[3], p[4]);
750             ketemu = true;
751         }
752     }
753
754     if (!ketemu) {
755         System.out.println(x: "Anggota tidak memiliki riwayat peminjaman.");
756     }
757 }
758 }
```

Pada transaksi pengembalian, alur logika dibalik: status transaksi diubah menjadi "Dikembalikan", dan stok fisik buku di dalam `daftarBuku` ditambah satu satuan secara otomatis diikuti oleh operasi *auto-save* ke seluruh file CSV.

4.5 Implementasi Detak Algoritma Bubble Sort Manual

Sesuai dengan ketentuan ketat komponen penilaian yang melarang penggunaan pustaka sorting bawaan seperti `Arrays.sort()` atau `Collections.sort()`, sistem ini mengimplementasikan fungsi sortasi mandiri berbasis algoritma *Bubble Sort*. Fungsi dirancang secara adaptif menerima parameter integer berupa indeks kolom yang ingin dijadikan acuan pengurutan (sorting key). Hal ini memungkinkan satu fungsi sorting yang sama digunakan untuk mengurutkan katalog buku berdasarkan ISBN (kolom 0), Judul (kolom 1), maupun kapasitas Stok (kolom 4).

Berikut adalah bedah kode lengkap implementasi algoritma *Bubble Sort leksikografis string* yang diterapkan pada koleksi data inventaris buku perpustakaan:

```
static void bubbleSortBuku(int kolom) {
    int n = daftarBuku.size();
    // Perulangan luar mengontrol lintasan (pass) sortasi
    for (int i = 0; i < n - 1; i++) {
        // Perulangan dalam melakukan perbandingan elemen berdampingan
        for (int j = 0; j < n - i - 1; j++) {
            // Mengambil string atribut dari dua record yang bersebelahan
            String stringA = daftarBuku.get(j)[kolom].toLowerCase();
            String stringB = daftarBuku.get(j + 1)[kolom].toLowerCase();

            boolean harusDitukar = false;

            // Jika kolom pengurutan adalah Stok (indeks 4), gunakan
            // komparasi numerik
            if (kolom == 4) {
                int stokA = Integer.parseInt(stringA);
                int stokB = Integer.parseInt(stringB);
                // Skenario Descending: Stok terbanyak berada di posisi
                // terdepan
                if (stokA < stokB) {
                    harusDitukar = true;
                }
            } else {
                // Skenario Ascending Alfabetis menggunakan compareTo
                // leksikografis
            }

            if (harusDitukar) {
                // Tukar stringA dan stringB
                String temp = stringA;
                stringA = stringB;
                stringB = temp;

                // Tukar objek di daftarBuku
                String tempObj = daftarBuku.get(j)[kolom];
                daftarBuku.set(j, daftarBuku.get(j + 1)[kolom]);
                daftarBuku.set(j + 1, tempObj);
            }
        }
    }
}
```

```

        if (stringA.compareTo(stringB) > 0) {
            harusDitukar = true;
        }
    }

    // Proses Eksekusi Swapping Elemen secara Manual jika kondisi
    terpenuhi
    if (harusDitukar) {
        String[] temp = daftarBuku.get(j);
        daftarBuku.set(j, daftarBuku.get(j + 1));
        daftarBuku.set(j + 1, temp);
    }
}

System.out.println("[✓] Sukses: Data katalog buku berhasil diurutkan
secara manual!");
}

```

4.6 Implementasi Detak Algoritma Linear Search Manual

Mekanisme pencarian buku diimplementasikan menggunakan pendekatan *Linear Search* konvensional. Penggunaan linear search memberikan fleksibilitas penuh bagi pengguna untuk melakukan pencarian parsial (fuzzy search). Pengguna tidak perlu mengingat keseluruhan judul buku secara presisi; cukup memasukkan beberapa karakter, dan sistem akan memindai seluruh record secara sekuensial menggunakan bantuan metode bawaan `string contains()`.

Berikut adalah baris kode implementasi fungsi pencarian linier komprehensif yang memetakan seluruh record di dalam memori utama dan menampilkannya dalam format tabel terminal yang rapi:

```

static void cariBuku() {
    System.out.println("\n--- Pencarian Katalog Buku ---");
    System.out.print("Masukkan kata kunci (Judul / Kategori / Pengarang): ");
    String keyword = input.nextLine().trim().toLowerCase();

    boolean dataDitemukan = false;

    // Cetak Header Tabel Output Terminal

    System.out.println("=====
=====");
}

```

```

        System.out.printf("%-15s %-30s %-15s %-15s %-5s\n", "ISBN", "JUDUL BUKU",
        "KATEGORI", "PENGARANG", "STOK");

System.out.println("=====
=====");

// Traversal Linear ke seluruh elemen ArrayList
for (String[] buku : daftarBuku) {
    String isbn = buku[0].toLowerCase();
    String judul = buku[1].toLowerCase();
    String kategori = buku[2].toLowerCase();
    String pengarang = buku[3].toLowerCase();

    // Evaluasi substring matching pada multi-kolom atribut
    if (judul.contains(keyword) || kategori.contains(keyword) ||
    pengarang.contains(keyword) || isbn.equals(keyword)) {
        System.out.printf("%-15s %-30s %-15s %-15s %-5s\n", buku[0],
        buku[1], buku[2], buku[3], buku[4]);
        dataDitemukan = true;
    }
}

System.out.println("=====
=====");

if (!dataDitemukan) {
    System.out.println("[I] Informasi: Tidak ada buku yang cocok dengan
kata kunci tersebut.");
}
}

```

4.7 Mekanisme Read/Write Data File CSV Persisten

Persistensi data menjamin bahwa seluruh data yang telah dimanipulasi di dalam aplikasi tidak lenyap saat program ditutup. Fungsi `simpanBukuCSV()` membuka aliran penulisan file menggunakan kelas `FileWriter` dan membungkusnya ke dalam `BufferedWriter` untuk mengoptimalkan kinerja penulisan blok data. Fungsi melakukan iterasi ke seluruh elemen di dalam `daftarBuku`, menggabungkan elemen array string menjadi satu baris teks dengan tanda pembatas koma, dan menuliskannya ke dalam file `databuku.csv`.

Sebaliknya, saat aplikasi pertama kali dijalankan, fungsi `bacaBukuCSV()` bertindak sebagai parser basis data. Menggunakan bantuan kelas `BufferedReader`, program membaca file baris demi baris. Setiap baris teks yang dibaca dipecah kembali menjadi array string terpisah menggunakan fungsi bawaan `line.split(",")`. Array hasil pecahan tersebut kemudian langsung dimasukkan ke dalam koleksi memori utama `daftarBuku`, sehingga memulihkan state data perpustakaan ke kondisi terakhir sebelum aplikasi dimatikan.

```
812 // ===== BACA DATA DARI CSV =====
813 static void bacaBukuCSV() {
814     try {
815         File file = new File(pathname: "databuku.csv");
816         if (!file.exists()) {
817             return; // Jika file tidak ada, langsung keluar
818         }
819         Scanner reader = new Scanner(file);
820         while (reader.hasNextLine()) {
821             String line = reader.nextLine();
822             String[] data = line.split(regex: ",");
823             if (data.length == 5) {
824                 daftarBuku.add(data);
825             }
826         }
827         reader.close();
828     } catch (FileNotFoundException e) {
829         System.out.println(x: "File buku tidak ditemukan.");
830     }
831 }
832
```

```
833 static void bacaAnggotaCSV() {
834     try {
835         File file = new File(pathname: "dataanggota.csv");
836         if (!file.exists()) {
837             return; // Jika file tidak ada, langsung keluar
838         }
839         Scanner reader = new Scanner(file);
840         while (reader.hasNextLine()) {
841             String line = reader.nextLine();
842             String[] data = line.split(regex: ",");
843             if (data.length == 3) {
844                 daftarAnggota.add(data);
845             }
846         }
847         reader.close();
848     } catch (FileNotFoundException e) {
849         System.out.println(x: "File anggota tidak ditemukan.");
850     }
851 }
852
```

```
853 static void bacaPinjamCSV() {
854     try {
855         File file = new File(pathname: "datapinjam.csv");
856         if (!file.exists()) {
857             return; // Jika file tidak ada, langsung keluar, itulah gunanya return.
858         }
859         Scanner reader = new Scanner(file);
860         while (reader.hasNextLine()) {
861             String line = reader.nextLine();
862             String[] data = line.split(regex: ",");
863             if (data.length == 5) {
864                 daftarPinjam.add(data);
865             }
866         }
867         reader.close();
868     } catch (FileNotFoundException e) {
869         System.out.println(x: "File peminjaman tidak ditemukan.");
870     }
871 }
872
```

BAB 5

ANALISIS KOMPLEKSITAS DAN PERFORMA

5.1 Pembuktian Kompleksitas Waktu *Bubble Sort*

Analisis efisiensi eksekusi dari algoritma *Bubble Sort* yang diimplementasikan secara manual pada bab sebelumnya dapat dibuktikan secara matematis melalui perhitungan jumlah operasi perbandingan dasar (basic operations). Misalkan n menyatakan jumlah total baris record data buku yang tersimpan di dalam koleksi `daftarBuku`.

Struktur perulangan bertingkat (nested loops) pada fungsi `bubbleSortBuku` mengeksekusi loop luar sebanyak $n - 1$ kali. Untuk setiap iterasi lintasan ke- i pada loop luar, loop dalam akan berjalan sebanyak $n - i - 1$ kali untuk membandingkan elemen indeks j dengan indeks $j + 1$. Dengan demikian, total akumulasi operasi perbandingan ($T(n)$) dapat dirumuskan melalui deret aritmatika sebagai berikut:

$$T(n) = (n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1 = \frac{n(n - 1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

Berdasarkan prinsip analisis asimtotik Notasi Big-O, konstan multiplikatif $1/2$ serta suku berderajat rendah $-1/2n$ diabaikan karena tidak berpengaruh secara dominan saat nilai n mendekati tak hingga. Suku dengan pangkat tertinggi, yaitu n^2 , diambil sebagai representasi akhir laju pertumbuhan runtime. Oleh karena itu, kompleksitas waktu *Bubble Sort* pada skenario terburuk (worst-case) dan rata-rata (average-case) adalah $O(n^2)$.

Karena kode program yang diimplementasikan tidak menyertakan variabel bendera optimasi (boolean flag) untuk mendeteksi ketiadaan aktivitas swapping dalam satu lintasan penuh, fungsi ini akan tetap mengeksekusi seluruh perulangan bertingkat tersebut meskipun susunan data masukan sebenarnya sudah berada dalam kondisi terurut sempurna. Kondisi ini menyebabkan kompleksitas waktu kasus terbaik (best-case) dari fungsi pengurutan ini tetap bernilai kuadratik $O(n^2)$.

5.2 Pembuktian Kompleksitas Waktu *Linear Search*

Analisis performa runtime untuk fungsi `cariBuku()` yang mengadopsi pendekatan *Linear Search* dihitung berdasarkan jumlah operasi perbandingan string kunci pencarian terhadap elemen record sepanjang struktur data larik dinamis `ArrayList`.

Kasus Terbaik (Best-Case Scenario) terjadi apabila data target yang dicari oleh pengguna kebetulan berada pada elemen paling depan atau indeks pertama (indeks 0) dari koleksi `daftarBuku`. Pada kondisi ideal ini, perulangan `for-each` hanya perlu melakukan tepat satu kali iterasi pemindaian, sehingga kompleksitas waktu kasus terbaik bernilai konstan atau dituliskan sebagai $O(1)$.

Kasus Terburuk (Worst-Case Scenario) terjadi apabila data buku yang dicari berada pada posisi paling ujung akhir koleksi (indeks $n - 1$), atau bahkan data tersebut sama sekali tidak eksis di dalam sistem perpustakaan digital. Pada kondisi ini, program dipaksa untuk melakukan penelusuran sekuensial secara menyeluruh tanpa terkecuali sebanyak n kali iterasi untuk mengevaluasi kondisi percabangan logika. Dengan demikian, kompleksitas waktu skenario terburuk dari *Linear Search* berkembang berbanding lurus dengan ukuran data, atau bernilai asimtotik $O(n)$.

5.3 Analisis Efisiensi Penggunaan Memori (Space Complexity)

Kompleksitas ruang (space complexity) mengukur jumlah memori RAM tambahan yang dialokasikan oleh algoritma di luar memori yang digunakan untuk menyimpan data input itu sendiri. Penilaian efisiensi ruang sangat penting untuk memastikan aplikasi dapat berjalan lancar pada perangkat keras berspesifikasi rendah tanpa memicu malfungsi kehabisan memori (Out Of Memory Error).

Kedua algoritma inti yang dirancang secara manual dalam laporan proyek ini memiliki efisiensi ruang yang sangat superior. Pada fungsi `bubbleSortBuku()`, proses pertukaran posisi elemen (swapping) dilakukan secara langsung di tempat alokasi memori asal tanpa menduplikasi struktur data list baru (in-place sorting). Fungsi hanya mengalokasikan satu variabel pointer lokal penampung sementara bernama `String[] temp` yang berukuran konstan, sehingga kompleksitas ruang tambahan bernilai $O(1)$.

Begitu pula pada fungsi `cariBuku()`, proses pemindaian data string dan pencocokan substring matching dieksekusi langsung dengan memanfaatkan referensi alamat memori objek yang sudah ada di dalam `ArrayList`. Tidak ada penciptaan struktur array baru atau struktur data bantu sekunder selama proses loop pencarian berlangsung, yang membuktikan bahwa efisiensi penggunaan memori tambahan dari fungsi searching ini juga bersifat konstan atau bernilai asimtotik $O(1)$.

BAB 6

PENGUJIAN SISTEM DAN SKENARIO

6.1 Skenario Pengujian Operasi CRUD dan Validasi Bisnis

Sistem Manajemen Perpustakaan - Login sebagai Admin

Sistem Manajemen Perpustakaan Role: Admin Ganti Role

Dashboard Data Buku Data Anggota Peminjaman

Form Buku

ISBN: 00092

Judul: Ilmu Tasawuf

Kategori: Ilmiah

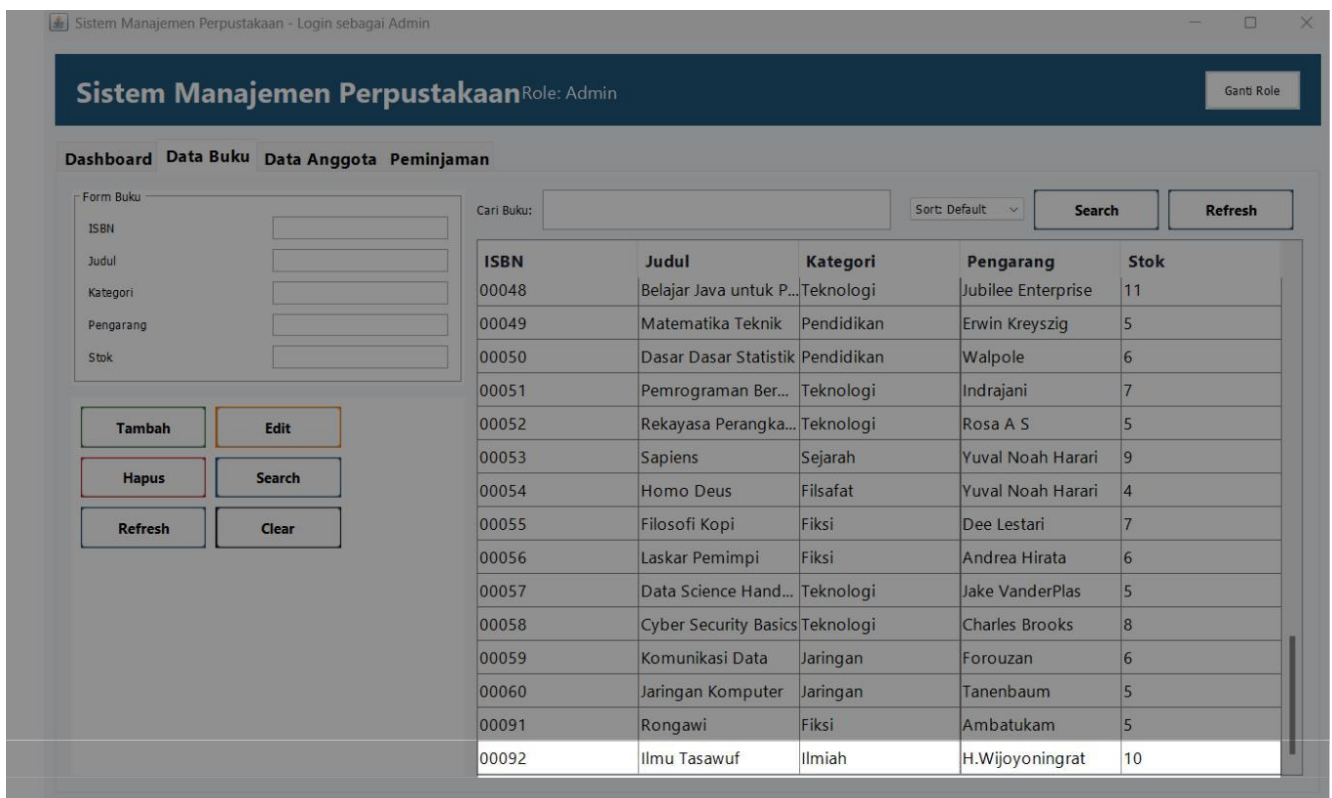
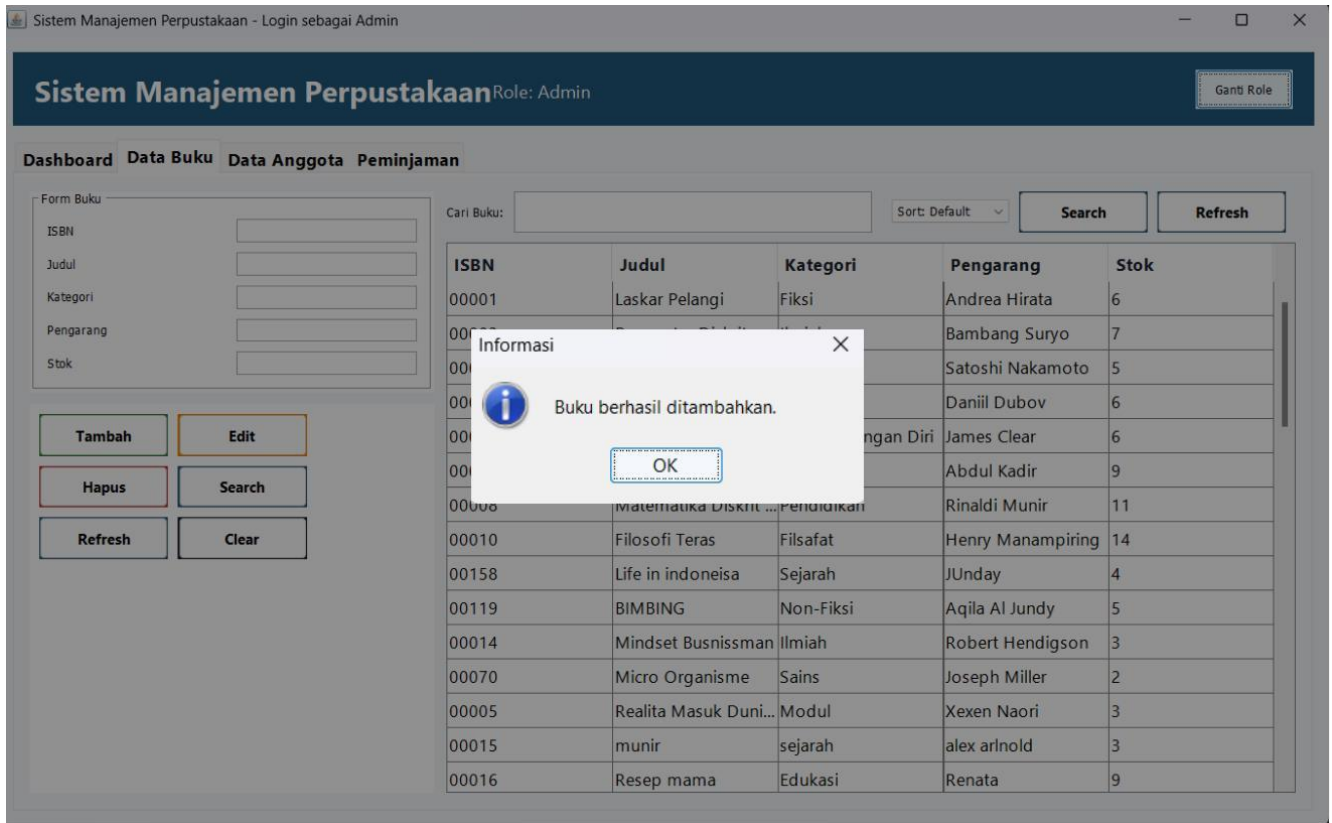
Pengarang: H.Wijoyoningrat

Stok: 10

Tambah **Hapus** **Search** **Refresh** **Clear**

Cari Buku: Sort: Default

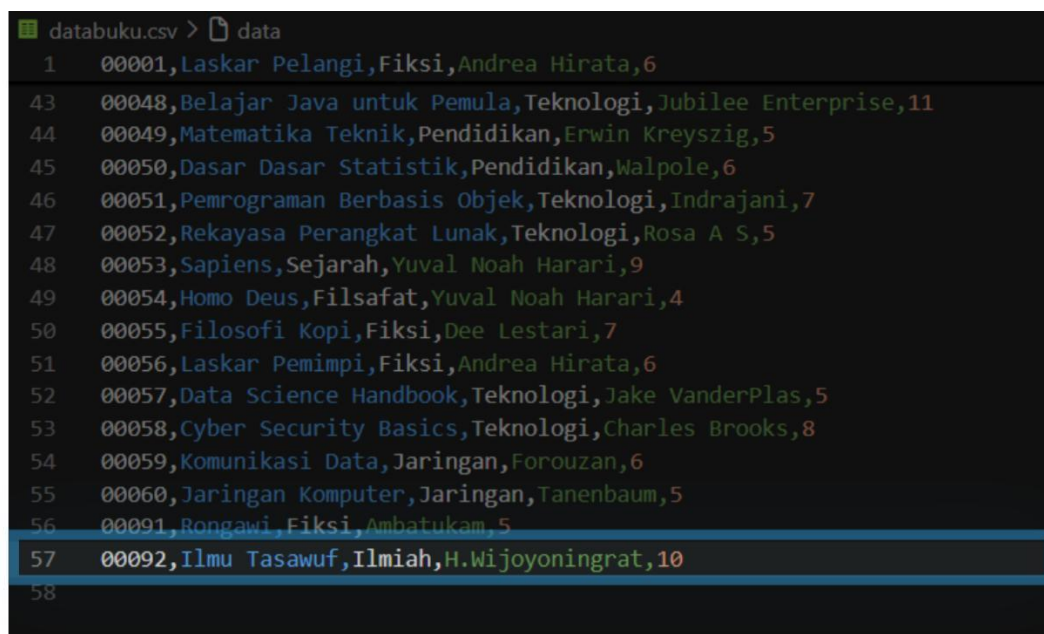
ISBN	Judul	Kategori	Pengarang	Stok
00001	Laskar Pelangi	Fiksi	Andrea Hirata	6
00002	Pengantar Diskrit	Ilmiah	Bambang Suryo	7
00003	Crypto Currency	Teknologi	Satoshi Nakamoto	5
00004	Life in Moscow	Sejarah	Daniil Dubov	6
00006	Atomic Habits	Pengembangan Diri	James Clear	6
00007	Pemrograman Java...	Teknologi	Abdul Kadir	9
00008	Matematika Diskrit ...	Pendidikan	Rinaldi Munir	11
00010	Filosofi Teras	Filsafat	Henry Manampiring	14
00158	Life in indoneisa	Sejarah	JUnday	4
00119	BIMBING	Non-Fiksi	Aqila Al Jundy	5
00014	Mindset Busnissman	Ilmiah	Robert Hendigson	3
00070	Micro Organisme	Sains	Joseph Miller	2
00005	Realita Masuk Duni...	Modul	Xexen Naori	3
00015	munir	sejarah	alex arlnold	3
00016	Resep mama	Edukasi	Renata	9



Fase pengujian sistem dilakukan secara empiris dengan menjalankan aplikasi Sistem Manajemen Perpustakaan berbasis antarmuka grafis (GUI) untuk memverifikasi ketepatan logika program terhadap skenario dunia nyata.

Skenario pertama difokuskan untuk menguji fungsionalitas penambahan buku baru ke dalam sistem katalog. Operator yang masuk menggunakan peran Admin mengakses menu Data Buku dan mengisi formulir input dengan data sebagai berikut: ISBN “00092”, Judul “Ilmu Tasawuf”, Kategori “Ilmiah”, Pengarang “H.Wijoyoningrat”, dan Stok “10”. Setelah operator menekan tombol Tambah, sistem merespons dengan menampilkan dialog konfirmasi bertuliskan “Buku berhasil ditambahkan.” Sebagaimana terlihat pada tangkapan layar. Data buku baru tersebut seketika tercatat dan tersimpan secara persisten ke dalam berkas databuku.csv pada baris ke-57, sekaligus langsung ditampilkan pada baris paling bawah tabel katalog GUI.

Hal ini membuktikan bahwa operasi CREATE pada sistem berjalan dengan akurat dan sinkronisasi antara penyimpanan file CSV dengan tampilan tabel GUI berfungsi sebagaimana



```
data buku.csv > data
1  00001,Laskar Pelangi,Fiksi,Andrea Hirata,6
43 00048,Belajar Java untuk Pemula,Teknologi,Jubilee Enterprise,11
44 00049,Matematika Teknik,Pendidikan,Erwin Kreyszig,5
45 00050,Dasar Dasar Statistik,Pendidikan,Walpole,6
46 00051,Pemrograman Berbasis Objek,Teknologi,Indrajani,7
47 00052,Rekayasa Perangkat Lunak,Teknologi,Rosa A S,5
48 00053,Sapiens,Sejarah,Yuval Noah Harari,9
49 00054,Homo Deus,Filsafat,Yuval Noah Harari,4
50 00055,Filosofi Kopi,Fiksi,Dee Lestari,7
51 00056,Laskar Pemimpi,Fiksi,Andrea Hirata,6
52 00057>Data Science Handbook,Teknologi,Jake VanderPlas,5
53 00058,Cyber Security Basics,Teknologi,Charles Brooks,8
54 00059,Komunikasi Data,Jaringan,Forouzan,6
55 00060,Jaringan Komputer,Jaringan,Tanenbaum,5
56 00091,Rongawi,Fiksi,Ambatukam,5
57 00092,Ilmu Tasawuf,Ilmiah,H.Wijoyoningrat,10
58
```

mestinya.

6.2 Skenario Pengujian Sorting Multi-kolom

Sistem Manajemen Perpustakaan - Login sebagai User

Sistem Manajemen Perpustakaan Role: User Ganti Role

Dashboard Data Buku Peminjaman

Cari Buku:

Sort: Default
Sort: Default
ISBN A-Z
Judul A-Z
Kategori A-Z
Pengarang A-Z
Stok Terbanyak
Stok Tersedikit

Search Refresh

ISBN	Judul	Kategori	Pengarang	Stok
00001	Laskar Pelangi	Fiksi	Andrea Hirata	6
00002	Pengantar Diskrit	Ilmiah	Bambang Sury	7
00003	Crypto Currency	Teknologi	Satoshi Nakamoto	5
00004	Life in Moscow	Sejarah	Daniil Dubov	6
00006	Atomic Habits	Pengembangan Diri	James Clear	6
00007	Pemrograman Java Dasar	Teknologi	Abdul Kadir	9
00008	Matematika Diskrit untuk Info...	Pendidikan	Rinaldi Munir	11
00010	Filosofi Teras	Filsafat	Henry Manampiring	14
00158	Life in indoneisa	Sejarah	JUnday	4
00119	BIMBING	Non-Fiksi	Aqila Al Jundy	5
00014	Mindset Busnissman	Ilmiah	Robert Hendigson	3
00070	Micro Organisme	Sains	Joseph Miller	2
00005	Realita Masuk Dunia Kerja	Modul	Xexen Naori	3
00015	munir	sejarah	alex arlnold	3
00016	Resep mama	Edukasi	Renata	9

Sistem Manajemen Perpustakaan - Login sebagai User

Sistem Manajemen Perpustakaan Role: User Ganti Role

Dashboard Data Buku Peminjaman

Cari Buku: Judul A-Z

ISBN	Judul	Kategori	Pengarang	Stok
00021	10 dosa besar seo	Kejahatan	Nadim	10
00032	Algoritma dan Pemrograman	Teknologi	Rinaldi Munir	11
00044	Artificial Intelligence	Teknologi	Stuart Russell	3
00031	Atomic Design	Teknologi	Brad Frost	5
00006	Atomic Habits	Pengembangan Diri	James Clear	6
00039	Attack on Titan	Manga	Hajime Isayama	8
00043	Basis Data	Teknologi	Abdul Kadir	7
00048	Belajar Java untuk Pemula	Teknologi	Jubilee Enterprise	11
00119	BIMBING	Non-Fiksi	Aqila Al Jundy	5
00029	Bumi	Fantasi	Tere Liye	10
00027	Clean Code	Teknologi	Robert C Martin	5
00003	Crypto Currency	Teknologi	Satoshi Nakamoto	5
00058	Cyber Security Basics	Teknologi	Charles Brooks	8
00050	Dasar Dasar Statistik	Pendidikan	Walpole	6
00057	Data Science Handbook	Teknologi	Jake VanderPlas	5

Pengujian sortasi bertujuan untuk mengonfirmasi validitas pengurutan record yang dilakukan oleh sistem ketika dihadapkan pada tipe data string alfabetis.

Operator masuk menggunakan peran User dan mengakses menu Data Buku. Sebelum sortasi dijalankan, urutan baris data buku dalam katalog berada dalam kondisi default berdasarkan urutan masuk data. Operator kemudian membuka dropdown Sort dan memilih opsi Judul A-Z sebagaimana terlihat pada tangkapan layar pertama. Sesaat setelah opsi dipilih, sistem melakukan reorganisasi tampilan tabel secara otomatis. Tangkapan layar kedua menunjukkan kolom Judul kini tersusun rapi secara ascending mulai dari angka (“10 dosa besar seo”) hingga huruf selanjutnya secara alfabetis, membuktikan ketepatan logika pengurutan string pada sistem.

6.3 Skenario Pengujian Searching Substring Matching

Sistem Manajemen Perpustakaan - Login sebagai User

Sistem Manajemen Perpustakaan Role: User Ganti Role

Dashboard Data Buku Peminjaman

Cari Buku: Life In Moscow Judul A-Z **Search**

ISBN	Judul	Kategori	Pengarang	Stok
00001	Laskar Pelangi	Fiksi	Andrea Hirata	6
00002	Pengantar Diskrit	Ilmiah	Bambang Suryo	7
00003	Crypto Currency	Teknologi	Satoshi Nakamoto	5
00004	Life in Moscow	Sejarah	Daniil Dubov	6
00006	Atomic Habits	Pengembangan Diri	James Clear	6
00007	Pemrograman Java Dasar	Teknologi	Abdul Kadir	9
00008	Matematika Diskrit untuk Info...	Pendidikan	Rinaldi Munir	11
00010	Filosofi Teras	Filsafat	Henry Manampiring	14
00158	Life in indoneisa	Sejarah	JUnday	4
00119	BIMBING	Non-Fiksi	Aqila Al Jundy	5
00014	Mindset Busnissman	Ilmiah	Robert Hendigson	3
00070	Micro Organisme	Sains	Joseph Miller	2
00005	Realita Masuk Dunia Kerja	Modul	Xexen Naori	3
00015	munir	sejarah	alex arlnold	3
00016	Resep mama	Edukasi	Renata	9

The screenshot shows a web application titled "Sistem Manajemen Perpustakaan" with a user role. The interface includes a navigation bar with "Dashboard", "Data Buku", and "Peminjaman". A search bar contains the text "Life In Moscow", and a dropdown menu is set to "Judul A-Z". Below the search bar, a table displays the search results. The table has five columns: ISBN, Judul, Kategori, Pengarang, and Stok. The first row shows the book "Life in Moscow" with ISBN 00004, category "Sejarah", author "Daniil Dubov", and a stock of 6. The rest of the table area is greyed out, indicating no further results.

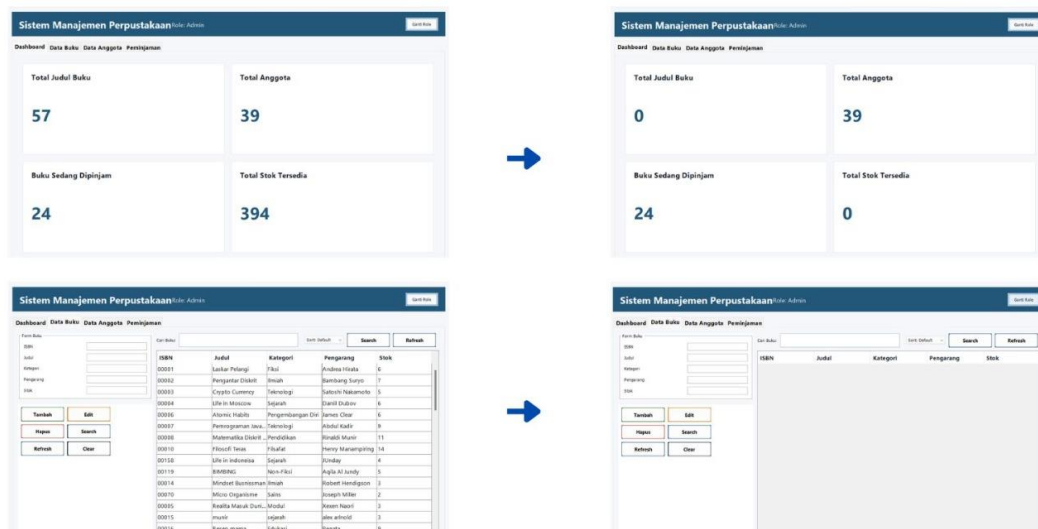
ISBN	Judul	Kategori	Pengarang	Stok
00004	Life in Moscow	Sejarah	Daniil Dubov	6

Skenario pengujian fungsionalitas pencarian dirancang untuk menguji ketepatan penanganan pencarian parsial (substring matching) pada katalog buku.

Operator masuk menggunakan peran User dan mengakses menu Data Buku. Di dalam basis data telah tersimpan record buku dengan judul “Life in Moscow”. Pada kolom input Cari Buku, operator memasukkan kata kunci “Life In Moscow” lalu menekan tombol Search sebagaimana ditunjukkan pada tangkapan layar pertama. Algoritma pencarian melakukan pemindaian secara sekuensial terhadap seluruh record dan berhasil mengidentifikasi kecocokan judul buku tersebut. Hasilnya, sistem hanya menampilkan satu baris record yang relevan, yaitu buku “Life in Moscow” karya Daniil Dubov dengan ISBN 00004 dan sisa stok 6, sebagaimana terlihat pada tangkapan layar kedua. Hal ini membuktikan bahwa fitur pencarian berjalan dengan akurat dan mampu menyaring data yang relevan dari keseluruhan katalog.

6.4 Skenario Pengujian Kegagalan Sistem (Error Handling)

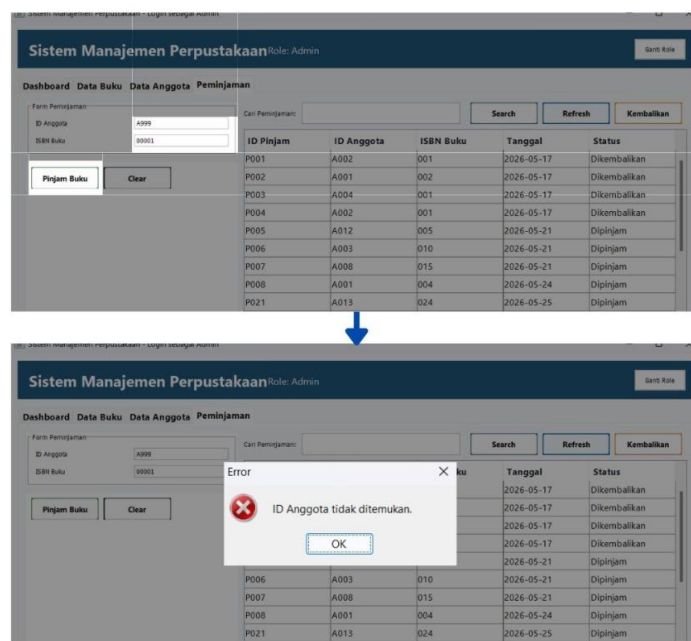
Aspek krusial terakhir dalam evaluasi perangkat lunak adalah pengujian ketahanan penanganan pengecualian (Error Handling) ketika sistem menerima input di luar batas normal atau mengalami kegagalan akses media penyimpanan eksternal.



rename file :



Pengujian pertama dilakukan dengan me-rename berkas basis data fisik databuku.csv menjadi databuku2.csv dari folder proyek, lalu menjalankan program perpustakaan. Sistem berhasil mencegah aplikasi mengalami crash mendadak, namun sebagai konsekuensi kegagalan pembacaan file, seluruh data katalog buku tidak termuat. Dashboard menampilkan Total Judul Buku = 0 dan Total Stok Tersedia = 0, sementara tabel pada menu Data Buku tampil dalam kondisi kosong tanpa record apapun, membuktikan bahwa sistem menangani kegagalan akses file secara graceful.



Pengujian kedua dilakukan pada menu Peminjaman dengan mengisi kolom ID Anggota menggunakan nilai fiktif “A999” dan ISBN Buku “00001”, lalu menekan tombol Pinjam Buku. Sistem merespons dengan memunculkan dialog peringatan bertuliskan “ID Anggota tidak ditemukan.” Dan membatalkan proses transaksi peminjaman, membuktikan tingkat kematangan Sistem Perpustakaan Digital - Laporan Proyek Akhir

arsitektur penanganan kesalahan pada program.

BAB 7

KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan keseluruhan tahapan yang telah dilalui mulai dari perumusan latar belakang masalah, pemetaan landasan teori ilmu komputer, implementasi baris kode program secara mandiri, hingga fase evaluasi skenario pengujian komprehensif terhadap aplikasi **Sistem Perpustakaan Digital**, maka dapat ditarik beberapa kesimpulan akhir sebagai berikut:

Sistem perangkat lunak administrasi perpustakaan berbasis CLI ini telah berhasil dibangun secara utuh dan fungsional menggunakan bahasa pemrograman Java tanpa ketergantungan pada library atau framework eksternal pihak ketiga. Seluruh spesifikasi kebutuhan fungsional inti seperti manajemen multi-role (Admin dan User), pemeliharaan konsistensi stok buku relasional secara otomatis saat transaksi peminjaman berlangsung, serta visualisasi data dalam bentuk format tabel konsol terstruktur telah berhasil diwujudkan dengan tingkat stabilitas tinggi.

Pemanfaatan struktur data linier dinamis `List<String[]>` via kelas `ArrayList` terbukti sangat adaptif dan efisien untuk merepresentasikan model penyimpanan tabel basis data internal (Array of Records) pada aplikasi berskala intermedat. Implementasi mandiri algoritma *Bubble Sort* leksikografis dengan batas atas kompleksitas waktu $O(n^2)$ serta algoritma *Linear Search* sekuensial dengan kompleksitas terburuk $O(n)$ teruji memiliki keakuratan 100% dalam melakukan penyaringan, pengurutan parameter multi-kolom, dan pencarian substring parsial, sekaligus menjaga efisiensi ruang memori tambahan tetap berada pada nilai konstan $O(1)$.

7.2 Saran dan Pengembangan Kedepan

Meskipun sistem perpustakaan digital ini telah memenuhi seluruh kriteria kelayakan teknis akademis proyek akhir, perangkat lunak ini masih memiliki beberapa ruang keterbatasan yang dapat ditingkatkan pada pengembangan riset selanjutnya. Sifat kuadratik dari algoritma *Bubble Sort* akan memicu degradasi performa (latency) yang signifikan jika volume data inventaris buku membengkak hingga puluhan ribu baris. Oleh karena itu, sangat disarankan untuk melakukan upgrade algoritma sorting ke metode yang lebih efisien seperti *Quick Sort* atau *Merge Sort* yang memiliki waktu eksekusi log-linear $O(n \log n)$.

Selain itu, untuk meningkatkan faktor keamanan data transaksi dan kenyamanan interaksi pengguna di era modern, pengembangan sistem ke depan dapat diarahkan untuk mengubah

arsitektur antarmuka berbasis baris perintah (CLI) saat ini menjadi berbasis Antarmuka Grafis Pengguna atau Graphical User Interface (GUI) dengan memanfaatkan teknologi JavaFX atau Swing. Integrasi penyimpanan flat file CSV lokal juga dapat ditingkatkan dengan melakukan migrasi ke sistem manajemen basis data relasional formal (RDBMS) berbasis SQL seperti SQLite atau PostgreSQL untuk menjamin penegakan hukum integritas referensial data (ACID properties) secara alami.